

datalab 报告

姓名：曾铭

学号：2025201706

总分	bitAnd	bitXor	samesign	logtwo	byteSwap	reverse	logicalShift	leftBitCount	float_i2f	floatScale2	float64_
37	1	1	2	4	4	3	3	4	4	4	3

test 截图：

```
dataLab2026 on } main [!] via C v15.2.0-gcc via v3.14.6
> python test.py
rm -f *.o btest fshow ishow *~ yacctab.py lextab.py result.txt .autograder_result
gcc -O0 -Wall -std=gnu99 -lm -o btest bits.c btest.c decl.c tests.c
gcc -O0 -Wall -std=gnu99 -o fshow fshow.c
gcc -O0 -Wall -std=gnu99 -o ishow ishow.c
Make success.
bitAnd      1/1:    PASS
bitXor      1/1:    PASS
samesign    2/2:    PASS
logtwo      4/4:    PASS
byteSwap    4/4:    PASS
reverse     3/3:    PASS
logicalShift 3/3:    PASS
leftBitCount 4/4:    PASS
float_i2f    4/4:    PASS
floatScale2  4/4:    PASS
float64_f2i  3/3:    PASS
floatPower2  4/4:    PASS
Total points: 37
```

解题报告

亮点

- 1. logtwo
- 2. float64_f2i

logtwo

思路

对于正整数 v ， $\text{logtwo}(v)$ 等于其二进制表示中最高位 1 的位置。
采用 $16 \rightarrow 8 \rightarrow 4 \rightarrow 2 \rightarrow 1$ 的二分定位方式寻找最高有效位，并用按位或组合结果。

伪代码：

```
ans = 0

依次检查 16、8、4、2、1：
    判断当前前半部分是否非零
    若非零：
        将对应的 16 / 8 / 4 / 2 / 1 记录到 ans
        将 v 右移相应位数

返回 ans
```

实现中，每次比较结果只有 0 或 1。例如第一步：

```
int step1 = (v >> 16) > 0;
int move1 = step1 << 4;
```

之后继续在已经缩小的范围内判断，使用 `|` 组合结果

```
int step2 = ((v >> move1) >> 8) > 0;
int move2 = (step2 << 3) | move1;
```

固定进行 5 轮判断即可确定最高有效位。

float64_f2i

思路

不完整保存 53 位有效数，只提取最终转换为 32 位整数时真正需要的高位。
其实经过前面的判断，只需要拼接这么多位数（向0舍入直接截断）。

伪代码：

```
读取 sign 和 exp
E = exp - Bias

if E < 0:
    return 0

if 超出 int 范围:
    return 0x80000000

构造有效数的高 32 位:
    加入规格化数默认的最高位 1
    接上 frac 中需要的部分

根据 E 右移得到整数部分

if sign 表示负数:
    对结果取补码

return result
```

frac拼接为：

```
unsigned frac = (1 << 31)
               | ((uf2 & 0x00FFFFFF) << 11)
               | ((uf1 & 0x7FF00000) >> 21);
```

可以直接根据实际指数 E 选择需要的位数：

```
res = frac >> (31 - E);
```

右移被舍弃的部分正好对应小数部分，因此自然实现向零取整。

参考的重要资料

PPT中求数字中1的个数的并行思想

[DataLab解析博客](#)主要有参考leftBitCount的思路